

 Devinterview-io / react-native-interview-questions



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)



React Native interview questions and answers to help you prepare for your next technical interview in 2025.

[devinterview.io/](#)

66 stars 10 forks 1 watching Branches Activity Tags

Public repository

main 1 Branch 0 Tags

Go to file

Go to file

Add file

Code

Devinterview-io 100 Must-Know React Native Interview Questions in 2025 36c3b13 · 6 months ago

README.md 100 Must-Know React Native Interview Questio... 6 months ago

100 Must-Know React Native Interview Questions in 2025

Prepping for a Web or Mobile Dev Interview?

Check out [Devinterview.io](#) for 5325+ questions covering 58 Full-Stack development topics

Kickstart your prep →

You can also find all 100 answers here [Devinterview.io - React Native](#)

1. What is *React Native* and how does it differ from *React*?

React Native extends the award-winning React library, making it possible to build **native mobile applications** using familiar web technologies.

Differences from React

- **Platform Scope:** React is tailored for web development, while React Native is exclusive to building iOS and Android applications.
- **Rendering Engine:** React uses the browser's DOM for visualization, whereas React Native achieves a parallel outcome through native platform rendering.
- **Component Style:** While most of the component-building strategies and lifecycles between React and React Native are analogous, the **controls manage** the considerable difference in rendering and event handling. For instance, React uses simple buttons and divs, whereas React Native leverages platform-compliant components like `Button`, `View`, and `Text`.
- **Integration with APIs:** While React targets web APIs, React Native consolidates connectivity with native mobile device features and APIs. This extension makes it feasible to tap into mechanism such as Camera, GPS, and Fingerprint sensors.

Code Example: React vs React Native

Here is the React code:

```
import React, { useState } from 'react';

const MyComponent = () => {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count+1)}>
      Clicked {count} times
    </button>
  );
};
```

```
    </button>
  );
};
```

And here is the equivalent React Native code:

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const MyComponent = () => {
  const [count, setCount] = useState(0);

  return (
    <View>
      <Button title={`Clicked ${count} times`} onPress={() => setCount(count+1)} />
    </View>
  );
};
```

2. Can you explain the concept of "Learn once, write anywhere" in the context of React Native?

The main idea of "Learn Once, Write Anywhere" in the context of **React Native** is "**code reusability**". Libraries and components written in **React** can be utilized across multiple platforms, while platform-specific code sections adapt as needed.

This approach offers **significant efficiency** in both development and maintenance as it ensures **consistent behavior** across platforms.

Key Concepts

- **Abstraction:** Developers can focus on business logic without being overly concerned about low-level platform intricacies.
- **Adaptation:** Platform-specific visual and user experience elements can be incorporated when necessary.
- **Reusable Components:** Leveraging shared components across platforms reduces redundancy and simplifies code maintenance.

Core Mechanisms

- **Platform Selectors:** Logical checks based on the running platform, such as `Platform.OS === 'ios'`, provide branching capabilities.
- **Platform-related Directories:** Platform-specific files can be organized into dedicated directories (e.g., `ios` and `android`), ensuring distinct settings and behaviors.
- **Platform-specific Extensions:** File naming conventions utilizing platform-specific extensions (e.g., `filename.ios.js`) enable tailored module imports.
- **Conditional Styles:** Styles imported or applied differently on distinct platforms ensure visual consistency.

Application Scenarios

- **Authenticators:** Different authentication workflows often exist for iOS and Android. The shared codebase can adapt gracefully using platform-specific file imports and methods.
- **UI/UX Adaptations:** When the look and feel of certain components deviate across platforms, such as navigational elements, shared business logic with platform-specific UI components is key.
- **Push Notifications:** Configurations and handling strategies for push notifications might necessitate platform-specific implementations.
- **Permissions:** The way permissions like location or camera are requested and handled can differ between **iOS** and **Android**, requiring adaptation within the common React Native codebase.

Caveats and Best Practices

- **Balance:** Strive for a harmonious blend of shared and platform-specific code, avoiding gridlock due to over customization.
- **Aim at Consistency:** Use platform-agnostic libraries as much as possible to ensure a uniform look and feel.
- **Maintenance Awareness:** Background differences in platform conventions, potential updates, and evolving third-party libraries underscore the need for periodic reviews of platform-specific modules.

3. How do you create a basic *React Native application*?

Creating a basic **React Native** application involves a number of steps, such as setting up your development environment, installing required software, and running a boilerplate application.

Setting Up Your Environment

First, ensure **Node.js** and **npm** are installed on your computer. To check for existing installations:

- For Node.js, use `node -v` or `npm -v`. If not installed, get the latest version from [Node.js website](https://nodejs.org/).
- For npm, use `npm -v`.

Setting Up Expo

Expo offers a quicker setup for a lightweight app. However, for large or complex applications, direct use of React Native might be more suitable. To set up Expo, install it globally via npm:

```
npm install -g expo-cli
```



Creating Your Project

With Expo (Recommended for Beginners)

1. Choose a Development Tool: You can select **Expo Go**, **simulator/emulator**, or **physical device** for testing.
2. Create a New Project: Use the `expo init` command and choose a template. Common choices include types like "blank," "tabs," and "from existing git repo."
3. Test Your Setup: Run "hello world" with `expo start`.

Without Expo

If you're not using Expo, create a new project with npm or Yarn:

With npm:

```
npx react-native init MyApp  
cd MyApp  
npx react-native run-android  
# OR  
npx react-native run-ios
```



With Yarn:

```
npx react-native init MyApp  
cd MyApp  
yarn android  
# OR  
yarn ios
```



Looking at Your Project

1. **app.json**: This file holds configuration settings.
2. **package.json** and **yarn.lock** (if using Yarn): These files manage project dependencies.
3. **node_modules/**: This directory stores your project's dependencies. This directory and its contents should never be pushed to your version control system.

App Entry Point

The default path for the app's entry point and main code is:

- For Android: `index.js` or `App.js` (generated by Expo)
- For iOS: `AppDelegate.m` and `main.m`

Directory Structure

The initial setup often includes these directories:

- **android/**: Contains the Android project.

- **ios/**: Contains the iOS project.
- **node_modules/**: Modules fetched and managed via npm or Yarn.
- **package.json**: Lists the app's metadata, as well as its dependencies.

4. What are *components* in *React Native*?

In React Native, **components** are building blocks that encapsulate UI and logic, making app development modular and efficient. There are two types of components: **Base Components** and **Custom Components**.

Base Components

These are core UI elements provided by React Native, directly corresponding to native views or controls. They are optimized for performance and interactive consistency.

- **Text**: Displays readable text.
- **View**: A container that supports layout with styles, such as flexbox.
- **Image**: Displays images.

Custom Components

These are created by developers and can be composed of both base and custom components, offering a higher level of abstraction. Custom components are reusable, promote a consistent design, and streamline UI updates.

Text_Example.jsx

Here is the React Native code:

```
import React from 'react';
import { Text } from 'react-native';

const CustomText = ({ children }) => (
  <Text style={{ fontFamily: 'Roboto-Bold', color: 'darkslategray' }}>{children}</Text>
);

export default CustomText;
```



Component Nesting and Tree Structure

React Native applications are tree-structured with multiple components nested within one another. This composition allows for consistent and quick alterations across the app. Whether it's a `Text`, `View`, or custom component, each is a node in the component tree, visually impacting the app. Devs split the UI into smaller, self-contained parts to simplify maintenance and testing.

🔗 Core Principles of Building Components

1. **Reusability**: Both base and custom components are designed for reuse in different parts of the application, further expanding the idea of modular development.
2. **Autonomy**: Each component should be self-sufficient, not heavily reliant on external data or functionality. This promotes easier maintenance and testing.
3. **UI Focus**: Components should either cater to UI or some specific functionality, but never both. This separation ensures a better code structure and maintainability.
4. **Loose Prop Types**: Custom components should generally avoid having too many mandatory props to allow for flexibility in their usage. They can, instead, rely on sensible defaults.
5. **Integrative Mindset**: When designing components, developers must have a holistic approach, keeping in mind how everything will come together in the UI.

Managing Component State

State management in components revolves around keeping track of changing data within that component. It's common in interactive UIs and involves data binding and conditional rendering. In React Native, components invoke a `useState` hook to integrate reactive state management.

CustomText with State

Here is the modified `CustomText` component:

```
import React, { useState } from 'react';
import { Text, TouchableOpacity } from 'react-native';

const CustomText = ({ children }) => {
  const [isBold, setIsBold] = useState(false);

  const toggleBold = () => setIsBold(prevState => !prevState);

  return (
    <TouchableOpacity onPress={toggleBold}>
      <Text style={{ fontFamily: isBold ? 'Roboto-Bold' : 'Roboto-Regular', color: 'darkslategray' }}>{children}</Text>
    </TouchableOpacity>
  );
};

export default CustomText;
```

5. Explain the purpose of the `render()` function in a *React Native component*.

The `render()` function, which is **mandatory** for all **React** and **React Native** components, is a gateway for JSX, receiving, processing, and returning the JSX layout. This function is like a workbench where the developer prepares the visual representation.

JSX: Visual Blueprint

JSX is HTML-like markup within JavaScript that provides a structured description of the visual layout. It's like a **visual blueprint** for the component.

README

Here's a simple example:

```
// JSX Blueprint
let myJSX = (
  <View>
    <Text>Hello, World!</Text>
  </View>
);

// 'render()' Function
let render = () => {
  let uiComponent = (
    <View>
      <Text>Hello, World!</Text>
    </View>
  );

  // Visual representation
  return uiComponent;
};
```

Code Maintenance

Having the `render()` function separates **declarative structure** from actual evaluatives and provides a clear workflow, making the code **easier to maintain** and understand.

Virtual DOM Interaction

React Native employs a **virtual DOM** to optimize and streamline UI updates. When the `state` or `props` of a component change, `render()` is called to ensure the **virtual DOM** is in sync. The virtual DOM then identifies and applies only the necessary updates to the actual UI, reducing redundancy and rendering time.

Performance Optimization: Conditional Rendering

Conditional rendering, controlled by `if-else`, is facilitated within the `render()` method, allowing for context-aware UI updates that ensure **sensible resource** and display utilization.

Side-Effects Handling: Lifecycle Methods

The `render()` method is just one of several **lifecycle methods**. Accurate handling of these methods through `render()` and controlled component updates ensures proper **data-fetching** and **side-effect management**.

UI Interactivity: Integrating JSX with Methods

JSX elements link visual representation with the logic behind user interactions—this is powered by methods like `onPress`, which, again, correspond to changes in `state` or `prop` triggers, leading back to, you guessed it, the trustworthy `render()` function.

Code Example: Using JSX and `render()`

Here is the React Native code:

```
import React, { Component } from 'react';
import { View, Text, TouchableOpacity, ScrollView } from 'react-native';

class MyComponent extends Component {
  state = {
    data: ['Item 1', 'Item 2', 'Item 3'],
    showList: true,
  };

  renderList() {
    if (!this.state.showList) {
      return null;
    }

    return (
      <ScrollView>
        {this.state.data.map((item, index) => (
          <Text key={index}>{item}</Text>
        ))}
      </ScrollView>
    );
  }

  toggleList = () => {
    this.setState({ showList: !this.state.showList });
  };

  render() {
    return (
      <View>
        <TouchableOpacity onPress={this.toggleList}>
          <Text>{this.state.showList ? 'Hide' : 'Show'} List</Text>
        </TouchableOpacity>
        {this.renderList()}
      </View>
    );
  }
}
```

6. What is *JSX* and how is it used in *React Native*?

JSX is a syntax extension for JavaScript, especially popular in React and React Native for expressing your UI components concisely.

It effectively lets you write **XML-style** code directly in your JavaScript files, making component definition and nesting visually intuitive.

Key JSX Features in React Native

- **Component Definition:** Use `.JSX` to visually group styling and component structure.

```
const MyComponent = () => {
  return (
    <View style={styles.container}>
      <Text>Hello, React Native!</Text>
    </View>
  );
};
```

- **Regular JS (No JSX):**

```
const MyComponent = () => {
  return React.createElement(View, { style: styles.container }, React.createElement(Text, null, "Hello, React Native!"));
};
```

- **Component Nesting:** Implement parent-child relationships via clear indentation in your JSX tree, enhancing visual hierarchy.

```
return (
  <View>
    <Text>Parent</Text>
    <View>
      <Text>Child 1</Text>
      <Text>Child 2</Text>
    </View>
  </View>
);
```

- **Event Handling:** Attach event listeners to UI events easily within the JSX structure.

```
<TouchableOpacity onPress={handlePress}>
  <Text style={styles.buttonText}>Press Me</Text>
</TouchableOpacity>
```

- **Equivalent Without JSX:**

```
React.createElement(TouchableOpacity, { onPress: handlePress }, React.createElement(Text, { style: styles.buttonText }, "Press Me"))
```

JSX Transpiling

The **Babel** transpiler lies at the heart of JSX functionality, converting JSX into regular JavaScript for compatibility with web and mobile platforms.

For instance, when you author this **React Native** code:

```
return <View style={styles.container}><Text>Hello, React Native!</Text></View>;
```

Babel transpiles it into the following JavaScript:

```
return React.createElement(View, { style: styles.container }, React.createElement(Text, null, "Hello, React Native!"));
```

7. Can you list some of the core components in *React Native*?

React Native has several fundamental components.

Core Components

1. **View:** The basic container that supports layout with Flexbox.
2. **Text:** For displaying text.
3. **Image:** For displaying images either from the local file system or the network.

Specialized Components

- **ScrollView:** For displaying a scrollable list of components.
- **Listview (deprecated):** A high-performance, cross-platform list view.
- **TextInput:** An input component with optional prompts, as well as a variety of keyboard types, enabling text input.

User Interface

- **Button:** A UI component that enables a user to interact with the application.
- **Picker:** A dropdown list that displays a picker interface.

Basic Functionality Components

- **ActivityIndicator**: Displays a rotating circle, indicating that the app is busy performing an operation.
- **Slider**: Lets the user select a value by sliding the thumb on the bar.
- **Switch**: Used for the on/off state.

iOS and Android Platform Integration

- **SegmentedControlIOS**: Renders a `UISegmentedControl` on iOS.
- **TabBarIOS**: Renders a tab bar with tabs that can be swiped.
 - `TabBarIOS.Item`: Represents an item in a `TabBarIOS` component.
- **ToolbarAndroid**: A toolbar for use with the `CoordinatorLayout`, offering additional features, such as controls for children of the Views.
 - `ToolbarAndroid`: Represents a standard Android toolbar.

List Views

- **FlatList**: A core virtualized list component supporting both vertical and horizontal scrolls. It's memory-efficient and only renders the elements on-screen. It also supports dynamic loading.
- **SectionList**: Much like `FlatList`, but also allows you to section your data.

Other Components

- **ActionSheetIOS**: Provides a pre-designed action sheet to display the list of options.
- **Alert**: For displaying an alert dialog.
- **AncestryManager**: Manages the relationships of Views. Developed initially for aiding the animation framework.
- **Animated**: A consolidated toolset to perform animations.
- **Appearance**: Gains insights into the unique appearance traits of respective iOS components if the device is running on iOS 13 or later.
- **DevSettings**: Owns a group of system settings tailored to the development phase.
- **Dimensions**: Assists in determining the dimensions of the viewport.
- **DormantStack**: Regulates the arrangement of Views. These views remain dormant and stop updating when they are not visible.
- **DrawerLayoutAndroid**: A unique drawer layout design developed for Android.
- **InputAccessoryView**: Accommodates customized views for input access.
- **KeyboardAvoidingView**: Augments the scalability of the UI in circumstances like keyboard being opened.
- **MaskedViewIOS**: Allows developers to create custom views with a specified shape.
- **Picker**: A seeming replica of the `Picker` that shows a native picker interface after a click.
- **Platform**: A utility to single out the present platform.
- **ProgressViewIOS**: For when one needs to display a standard iOS progress bar.
- **RefreshControl**: A tool used to integrate swipe-down-the-screen functionalities.
- **SafeAreaView**: A safety net component for iOS, regulating the area displays that would have compromised otherwise.
- **StatusBar**: A utility to handle the app's status bar control.
- **StyleSheet**: A collection of type-safe approaches to the integration of styles in your app.
- **TouchableHighlight**: A view that illustrates a sub-view while being tapped.
- **TouchableNativeFeedback**: An optimized version of the `TouchableHighlight` toward Android platforms.
- **TouchableOpacity**: Reduces the opacity of the view for conveying the touch response.
- **TouchableWithoutFeedback**: A view reacting to touches and signals exclusively without providing any feedback.
- **Vibration**: Triggers the device's vibration mechanism.
- **ViewPropTypes**: Manages the prop-types for the `View` component.

8. How do you handle *state management* in *React Native*?

State management in **React Native** involves tracking and updating the state of components.

Local State Management

- **State Declaration**: Use `useState` from the React library.

```
import React, { useState } from 'react';

const MyComponent = () => {
  const [count, setCount] = useState(0);
};
```



- **State Update:** Invoke the state-modifying function (in this case, `setCount`).

Global State Management

- **State Declaration:** Utilize the "Context" API via `Provider` to make the state globally available.
- **Selecting:** Components can subscribe to specific parts of the global state using `useContext` .
- **Updating:** To modify global state, make use of a "reducer" or actions that the `dispatch` function sends to the state management system.

```
import { useReducer, useContext } from 'react';

// Define an action
const INCREMENT = 'increment';

// Define a reducer
const reducer = (state, action) => {
  switch (action.type) {
    case INCREMENT:
      return { ...state, count: state.count + 1 };
    default:
      return state;
  }
};

// Within component
const MyComponent = () => {
  const { state, dispatch } = useContext(MyContext);
  dispatch({ type: INCREMENT });
};
```

External State Management

- **State Declaration:** Use external libraries like **Redux**.
- **Selecting:** Components can subscribe to parts of the global state via `connect` or `useSelector` .
- **Updating:** Trigger changes using `dispatch` or action creators.

Shared State Management

- **State Declaration:** Also facilitated by the "Context" API or libraries like **Redux**.
- **Updating:** Components interact with the shared state as per the framework's methodology, be it by using `dispatch` in **Redux** or other means.
- **Keeping Components in Sync:** This is automatically handled by the framework.

Best Practices

- **Granularity of State:** Prefer local state whenever state is limited to a single component. Global and external state management should be reserved for state shared across multiple components.
- **Consistency:** Adhere to a consistent state management approach throughout the application to minimize complexity.
- **Minimizing Storage:** Avoid duplicative storage of state where local state can serve the purpose.

9. What are *props* in *React Native* and how are they used?

Props (short for "properties") enable unidirectional data flow in React and are essential for building reusable **React Native components**.

Props provide a mechanism for passing data from a parent to a child, allowing for customization and dynamic behavior. They are immutable and help to keep components self-contained, making it easier to manage and maintain a React Native application.

Key Concepts

- **Unidirectional Data Flow:** Props serve as a one-way street for data, originating in a parent component and flowing down to child components.
- **Read-Only:** Once defined, props are not meant to be modified by the receiving component.
- **Default Values:** You can define default values for props to ensure smooth handling.

Guiding Principles

- **Single Source of Truth:** Emphasizes a centralized role for data, making it clearer to identify the information's origination point.
- **Separation of Concerns:** By restricting the scope in which components can influence one another, application logic becomes more compartmentalized and easier to manage.

Prop Types

- **Required Props:** Ensures that specific props are provided. If a required prop is missing, React will issue a warning (in development mode).
- **Data Types:** Prop types can be specialized (e.g., string, number, or function) to enforce type coherency, contributing to better code reliability and predictability.
- **Shape and Arrays:** For more complex data structures, such as objects with specific shapes and arrays, you can define even more intricate prop type requirements.

Code Example: Simple "Weather Card" Component

```
// WeatherCard.js
import React from 'react';
import PropTypes from 'prop-types';
import { Text, View, Image } from 'react-native';

const WeatherCard = ({ temperature, humidity, icon }) => {
  return (
    <View>
      <Text>`Temperature: ${temperature}`</Text>
      <Text>`Humidity: ${humidity}`</Text>
      <Image source={icon} />
    </View>
  );
};

WeatherCard.propTypes = {
  temperature: PropTypes.number.isRequired,
  humidity: PropTypes.number.isRequired,
  icon: PropTypes.shape({
    uri: PropTypes.string,
  }).isRequired,
};

export default WeatherCard;
```

In the above example, `WeatherCard` accepts three props: `temperature`, `humidity`, and `icon`. Each of these props is essential for the card to accurately depict current weather conditions. Each **prop has a designated prop type**, ensuring that the received values match specific criteria.

10. What is the significance of the *Flexbox* layout in *React Native*?

Let's look at the unique implementations and advantages of **Flexbox** within **React Native**.

Components Designed for Flexbox

React Native provides specific components empowered by Flexbox, including:

1. **Container Components:** These are Views and Touchables that house inner elements being placed using Flexbox.
2. **Content Components:** These are core layout components that handle the arrangement of inner items. Examples include the Text component.

Core Flexbox Components

1. **View:** The foundation of Flexbox layout in React Native
2. **Text:** A specialized View primarily for text-related elements that supports Flexbox
3. **Image:** A Flexbox-capable View for image elements
4. **ScrollView:** A container for components that are larger than its size, offering various scrolling methods. It uses Flexbox to arrange these scrollable components.
5. **FlatList** and **SectionList:** These specialized components efficiently render large lists and areas as per the screen's dimensions, leveraging the innate performance of the native interface.

6. **VirtualizedList**: A low-level, high-performance list. Both **FlatList** and **SectionList** are built on top of **VirtualizedList**.

Flexbox Properties in React Native

1. **Direction**: Establishes the principal axis of the layout. Options are `row` and `column`, with the latter being the default.
2. **Alignment**: Determines the position of items along the secondary axis. Common settings include `flex-start`, `center`, and `flex-end`. The setting `stretch` is also available, which extends the components to fill the empty space.
3. **Order**: Flexbox allows for reordering of elements. This property defines the display order, **with the default being 0**.
4. **Proportional Sizing**: Rather than specifying explicit dimensions, items can be sized proportionally to the remaining space, making the layout adaptable to various screen sizes.
5. **Gutters**: Flexbox in React Native can handle gutters between items effortlessly.

Code Example: Flex Direction

Here is the JavaScript code:

```
export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.box} />
      <View style={styles.box} />
      <View style={styles.box} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'row',
    justifyContent: 'space-evenly',
    alignItems: 'center',
  },
  box: {
    width: 50,
    height: 50,
    backgroundColor: 'lightgrey',
  },
});
```

In this example, the three boxes are displayed side by side due to `flexDirection: 'row'`. Their alignment is managed by `justifyContent` and `alignItems`.

React Native primarily focuses on the core concepts and properties of Flexbox, offering a simplified and intuitive experience for developers.

11. How do you debug a *React Native application*?

Debugging in React Native involves both traditional debugging methods and those specific to the platform. Here are the strategies and tools to streamline the debugging process.

Debugging Strategies

- **Use DevTools**: These browser-based tools let you inspect and modify the DOM, examine network activity, and monitor console logs, among other features.
- **Leverage Emulators/Simulators**: Virtual device environments can be invaluable for replicating and debugging issues across a range of device types.
- **Physical Testing**: Testing on actual devices is vital for identifying device-specific bugs. Use USB debugging for a direct connection to the development machine.
- **Watch Out for Undefined Behaviors**: JavaScript doesn't throw errors when accessing properties or methods of `null` or `undefined`. Check for unexpected `undefined` values using defensive patterns, like short-circuit evaluation or optional chaining.
- **Separate and Simplify Issues**: If you're dealing with a complex issue, break it down into bite-sized problems. Simplify the problem domain to understand the root cause. Use binary search principles to isolate issues methodically.

- **Test Isolation:** Confirm whether the bug is reproducible in a fresh environment. Temporarily remove certain components or functionality to see how it affects the bug.
- **Incremental Building:** After making a change, observe if the problem resolves or worsens.
- **Versatile Console Logging:** Consider using `console.assert` for conditional logs, `console.info` for informational messages, `console.warn` for potential issues, and `console.error` for errors.
- **Component-Specific Debugging:** Use `ReactDOM.findDOMNode` to fetch a component's DOM representation.
- **Debug in Multiple Environments:** It might be helpful to test your app in both dev and production modes.

Tools for Debugging

1. **React DevTools:** Browser extensions like Chrome DevTools or standalone applications facilitate inspection of React component trees, context, and hooks.
2. **Flipper:** This is a debugging tool for mobile apps on iOS, Android, and React Native. It offers a pluggable architecture that allows developers to build custom integrations.
3. **Expo:** The `expo-dev-client` package provides a powerful testing environment. It's useful when working in managed Expo projects.
4. **Redux DevTools:** For an app integrated with Redux, this tool offers essential features such as time-travel debugging.
5. **Network Inspectors:** Both Chrome DevTools and Safari's Web Inspector offer detailed network activity insights.
6. **Profiling and Performance Tools:** Use React DevTools for profiling and diving deep into performance metrics.
7. **Remote Debugging:** Chrome and Safari offer remote debugging tools. For example, with Chrome, you can inspect the WebView used by your app.
8. **React Native Debugger:** This application bundles Chrome DevTools, the React DevTools, and a JavaScript VM.
9. **Code Linters:** Integrate linters like ESLint or TypeScript with your editor or development environment to catch potential source code issues.
10. **HMR (Hot Module Replacement):** HMR ensures that your app remains in the current state whenever you make changes, reducing the need for constant refreshes.
11. **Third-Party Services:** Incorporate third-party services for bug tracking, crash reporting, and in-app user feedback.
12. **Continuous Integration and Continuous Deployment (CI/CD) Tools:** Platforms like Bitrise or CircleCI offer extensive pipelines for automation, testing, and deployment.

12. Explain the concept of *hot reloading* in *React Native*.

Hot Reloading innovatively speeds up the development process by rendering updates to the running app in real-time. While preserving the app's current state, it allows developers to observe changes in layout, live.

Key Benefits

- **Enhanced Productivity:** Avoiding repetitive recompilations and reinstalls.
- **Quick Updates:** View real-time changes with contextual data.
- **Error Localization:** Identify issues when and where they occur.

Mechanism

- **State Retention:** Unlike a full reload, hot reloading keeps the app's state intact.
- **Partial Update:** Only files that have been changed are updated, reducing the time needed.

Code Example: Enabling Hot Reloading

In `App.js`, activate hot reloading with a few simple changes:

```
if (__DEV__) {
  const { activateKeepAwake } = require('expo-keep-awake');
  activateKeepAwake();
}
```



13. How do you handle user input in *React Native*?

React Native comprises several components tailored to user interaction and input processing for both iOS and Android platforms.

Core Input Components

Text Input

The most basic input component, `TextInput`, caters to single- and multi-line text input needs. It provides options for simple text, password fields, and more.

Example: Basic `TextInput`

Here is the React Native code:

```
<TextInput placeholder="Enter your name" />
```



Input Fields with Auto-Complete

`TextInput` leverages `autoCompleteType` to blend with both platforms' keyboard suggestions.

If a user has previously entered data in the same field, the keyboard might suggest this data. On some platforms, this will include email address suggestions.

Choose one of the following:

- `off` : No auto-completion.
- `username` : Auto-completion for usernames.
- `password` : Secure auto-completion for passwords.
- `email` : Auto-completion for email addresses.

For example:

```
<TextInput  
  placeholder="Enter your email"  
  autoCompleteType="email"  
/>
```



Discrete options with Picker

For small sets of options, `Picker` macOS. `Picker` is the component that iOS makes available that allows for single option selection (much like a dropdown).

- `selectedValue` : The currently selected value.
- `onValueChange` : The event triggered when a different value is selected.

The iOS code is as follows:

```
<Picker  
  selectedValue={selectedValue}  
  onValueChange={({itemValue, itemIndex}) =>  
    setSelectedValue(itemValue)  
  }  
>  
  <Picker.Item label="Java" value="java" />  
  <Picker.Item label="JavaScript" value="js" />  
</Picker>
```



The android code is like:

```
<Picker  
  selectedValue={selectedValue}  
  onValueChange={({itemValue, itemIndex}) =>  
    setSelectedValue(itemValue)  
  }  
>  
  <Picker.Item label="Java" value="java" />
```



```
<Picker.Item label="JavaScript" value="js" />
</Picker>
```

Text Areas for Multi-Line Input

`TextArea` offers a multi-line text input field.

Here is the React Native code:

```
<TextArea placeholder="Type your message" />
```



14. What is a `TouchableHighlight` in *React Native*?

`TouchableHighlight` is a **React Native** component optimized for touch interactions. It uses the platform's native feedback effect when touched, making it ideal for buttons, tabs, or other interactive elements.

Key Features

- **Accessibility:** It automatically handles accessibility states such as focus and press, adhering to **WAI-ARIA** and native mobile accessibility guidelines.
- **Visual Feedback:** Upon touch, it provides a visual indication like opacity changes or highlighting, depending on the platform.
- **On Press Event:** Executes a function when the component is pressed or activated using keyboard or assistive devices.

Code Example: `TouchableHighlight`

Here is the React Native code:

```
import { TouchableHighlight, Text, View } from 'react-native';

const CustomButton = ({ label }) => (
  <TouchableHighlight
    style={{ backgroundColor: 'green', padding: 10, margin: 10, borderRadius: 5 }}
    underlayColor="lime"
    onPress={() => alert('Button pressed!')}
  >
    <Text style={{ color: 'white' }}>{label}</Text>
  </TouchableHighlight>
);

const App = () => (
  <View>
    <CustomButton label="Press me!" />
  </View>
);
```



15. Describe the *View* component and its purpose in *React Native*.

`View` in React Native functions similarly to a `<div>` in web development and allows for **style definitions**.

Core Benefits

- **Layout Flexibility:** Optimizes for different screen sizes and orientations.
- **Style Configuration:** Streamlines style organization and replication.

Primary Use-Cases

1. **Structural Containers:** Serve as building blocks for organizing UI elements.
2. **Standalone Components:** When you need an element that can be styled or requires touch interaction.
3. **Adaptive Design:** Design dynamic layouts that accommodate various device types and orientations.
4. **Styling:** Apply shared styles to multiple child components.

Common Misconceptions

1. View Doesn't Represent Visual Elements

While **View** forms the basic unit for layout and styling, it doesn't intrinsically display any content, nor is it scrollable. Within **View**, you incorporate **Text** and other components for visual representation.

2. View Doesn't Guarantee a Unique End-Component

Despite being fundamental for rendering, **View** might not directly translate to a distinct UI component. The internal view system structures UI arrangements. Accessibility and debug tools in React Native App recognize and isolate View components. This distinction is crucial for device adaptation and adopting universal design principles.

Best Practices

- **Organized Nesting:** Maintain clarity by thoughtfully nesting components.
- **Styling Economics:** Leverage shared styles to minimize redundancy and ensure a consistent look throughout the app.
- **Accessibility:** Maintain good accessibility by including visual aid for differently-abled users.
- **Refactoring:** Regularly reevaluate your design choices and refactor as needed.

Explore all 100 answers here 🙌 [Devinterview.io - React Native](#)

Prepping for a Web or Mobile Dev Interview?

Releases

No releases published

Packages

No packages published